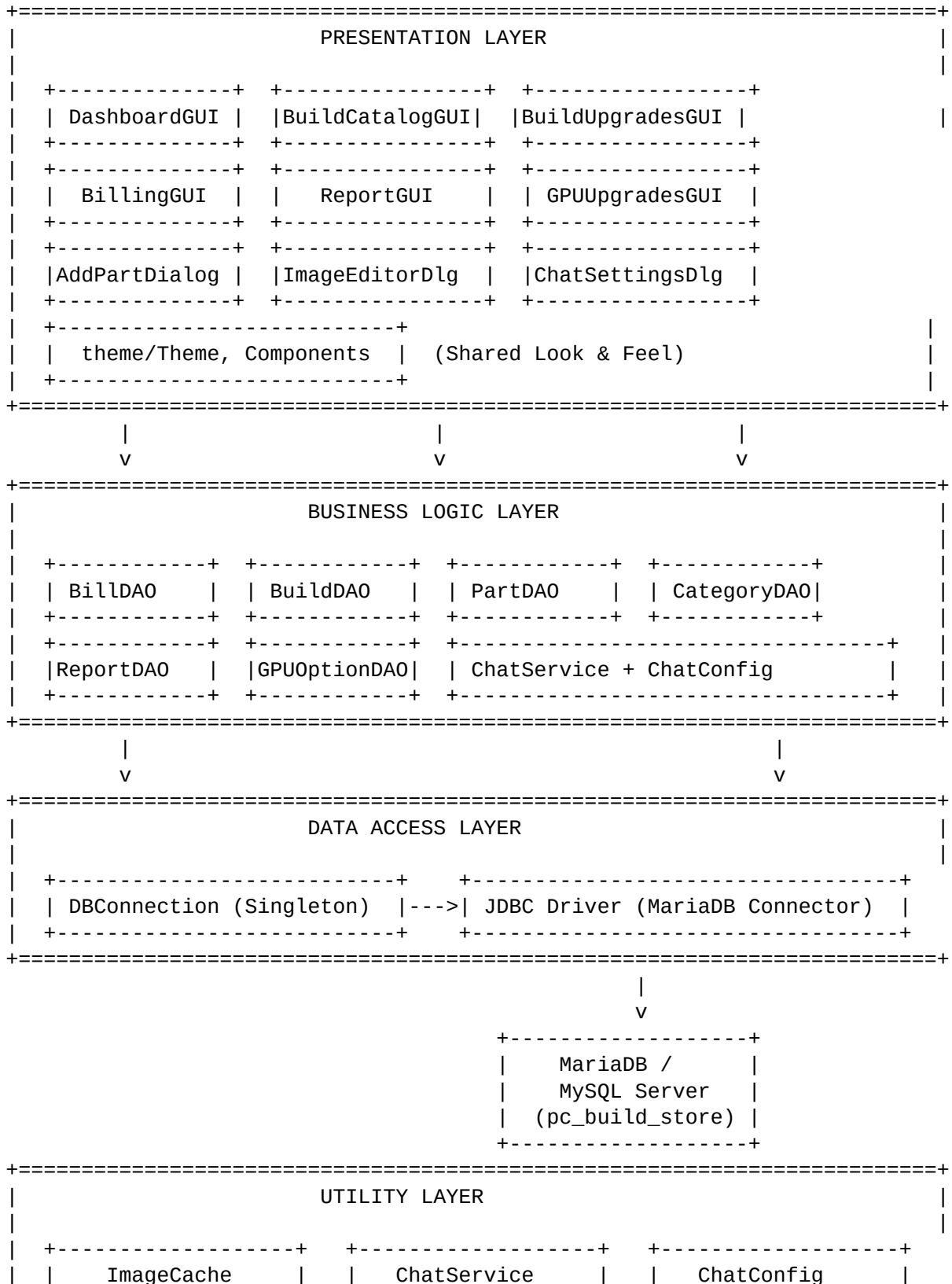


SYSTEM ARCHITECTURE - PC Build Store

The PC Build Store application follows a layered architecture pattern that separates concerns into five distinct layers. Each layer has a well-defined responsibility and communicates only with its adjacent layers, ensuring loose coupling and high cohesion.

ARCHITECTURE DIAGRAM



(memory + disk cache, batch loading)	(OpenAI streaming HTTP client)	(API key, model, prompt config)
+-----+	+-----+	+-----+
=====		
=====		
MODEL LAYER		
+-----+	+-----+	+-----+
Bill	Build	Part
+-----+	+-----+	+-----+
+-----+		
GPUOption		
+-----+		
=====		

LAYER DESCRIPTIONS

1. PRESENTATION LAYER

Package: com.pcbuildstore.ui

Technology: Java Swing

Responsible for all user interface rendering and user interaction. Each screen of the application is implemented as a separate JFrame or JPanel.

Components:

- DashboardGUI - Main menu with navigation buttons and summary cards (total builds, parts, bills, revenue). Entry point after login.
- BuildCatalogGUI - Browse and manage PC builds. Displays build cards with price, score, and part count. Supports creating, deleting, and selecting builds for editing.
- BuildUpgradesGUI - View and manage parts assigned to a build. Shows parts by category with prices and scores. Allows adding/removing parts and recalculating totals.
- BillingGUI - Generate bills from builds. Displays bill history with purchase dates, final prices, and build names. Supports viewing and deleting bills.
- ReportGUI - Analytics dashboard with bar charts showing revenue by build, builds per category, parts usage by brand, and price/score distributions.
- GPUUpgradesGUI - Suggests GPU upgrade paths for a selected build. Shows price and performance deltas for alternative GPUs that fit the build's budget tier.
- AddPartDialog - Modal dialog for adding new parts to the inventory. Dynamically shows/hides fields based on the selected category.
- ImageEditorDialog - Dialog for editing part image URLs. Validates URLs and provides preview.
- ChatSettingsDialog - Configuration dialog for the AI chat feature. Allows setting API key, model, and base URL.
- Theme/Components - Centralized styling utilities. Theme defines colors, fonts, and spacing. Components provides factory methods for styled buttons, labels, and panels.

2. BUSINESS LOGIC LAYER

Package: com.pcbuildstore.dao

Technology: Plain Java

Data Access Objects (DAOs) encapsulate all database operations. Each DAO maps database tables to model objects and provides CRUD operations plus

aggregate queries. DAOs handle:

- SQL query construction and execution
- ResultSet mapping to POJOs
- Connection lifecycle (obtaining from DBConnection singleton)
- Exception handling and logging

Components:

- | | |
|--------------|--|
| BillDAO | - CRUD for bills. Aggregates: totalBills, totalRevenue, highestBill, recentBills. |
| BuildDAO | - CRUD for builds and build_parts. Aggregates: averageScore, highestPricedBuild, highestScoreBuild, recentBuilds. Handles part-to-build assignment with ON DUPLICATE KEY UPDATE for upserts. |
| PartDAO | - CRUD for parts. Compatibility queries: getCompatibleParts (by DDR), getCompatibleCPUs (by socket), getAdequatePSUs (by wattage). Aggregates: partCount, averagePrice, mostExpensive. |
| CategoryDAO | - Read-only access to categories. Lookup by ID or name. |
| GPUOptionDAO | - CRUD for GPU upgrade options. Query by budget tier. |
| ReportDAO | - Read-only aggregate queries for analytics: buildsPerCategory, revenueByBuild, buildPartDetails, partsUsageByBrand, priceDistribution, scoreDistribution. |
| ChatService | - HTTP client for OpenAI-compatible streaming APIs. Uses Java HttpClient with SSE parsing. Runs on a daemon thread to keep the UI responsive. |

3. UTILITY LAYER

Package: com.pcbuildstore.util, com.pcbuildstore.chat

Technology: Java standard library

- | | |
|------------|---|
| ImageCache | - Multi-tier image caching system: <ul style="list-style-type: none">L1: In-memory LRU cache (120 entries, LinkedHashMap)L2: Disk cache (~/.pcbuidstore/cache/images/)L3: HTTP download with fallback defaults Features: async batch loading with ExecutorService, category-colored placeholder images, disk persistence. |
| ChatConfig | - Manages AI chat configuration. Persists to/from ~/.pcbuidstore/config.json. Provides validation and URL construction for the chat completions API. |

4. DATA ACCESS LAYER

Package: com.pcbuildstore.database

Technology: JDBC + MariaDB Connector/J

- | | |
|--------------|--|
| DBConnection | - Singleton connection manager. Maintains a single persistent JDBC connection. Provides automatic reconnection if the connection is closed or null. Connects to localhost:3306/pc_build_store. |
|--------------|--|

5. MODEL LAYER

Package: com.pcbuildstore.models

Technology: Plain Java POJOs

Simple data transfer objects with private fields, constructors, and getters. No business logic - purely structural representations of database rows.

Components:

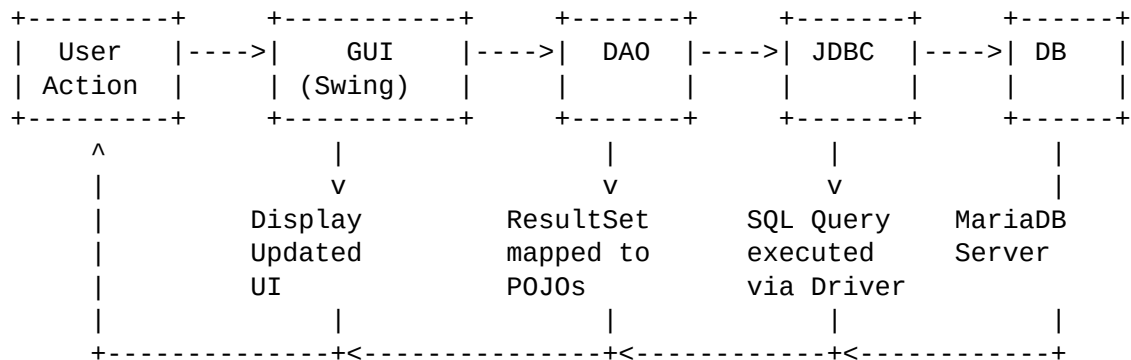
- | | |
|-----------|---|
| Bill | - billId, buildId, finalPrice, finalScore, purchaseDate, buildName (denormalized from JOIN) |
| Build | - buildId, name, totalPrice, totalScore, createdAt |
| BuildPart | - id, buildId, categoryId, partId, priceAtAdd, categoryName, partName, partBrand (denormalized) |
| Category | - categoryId, name |

Part	- partId, categoryId, brand, name, price, performanceScore, socketType, ddrGeneration, coreCount, clockSpeed, vram, memorySpeed, capacity, readSpeed, wattage, efficiency, imagePath
GPUOption	- gpuOptionId, gpuPartId, forBudget, priceIncrease, performanceIncrease, gpuName, gpuBrand (denormalized)

DATA FLOW

The application follows a synchronous request-response data flow for database operations and an asynchronous pattern for image loading and AI chat streaming.

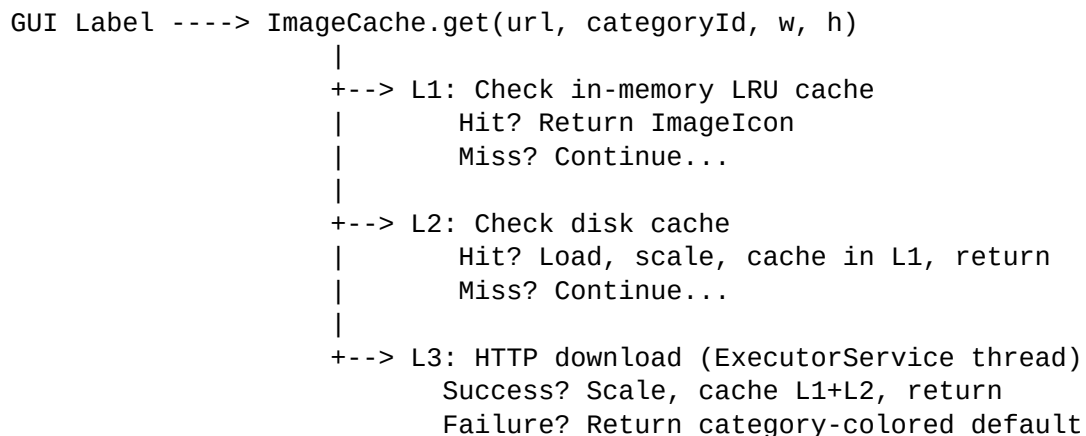
PRIMARY DATA FLOW (Database Operations):



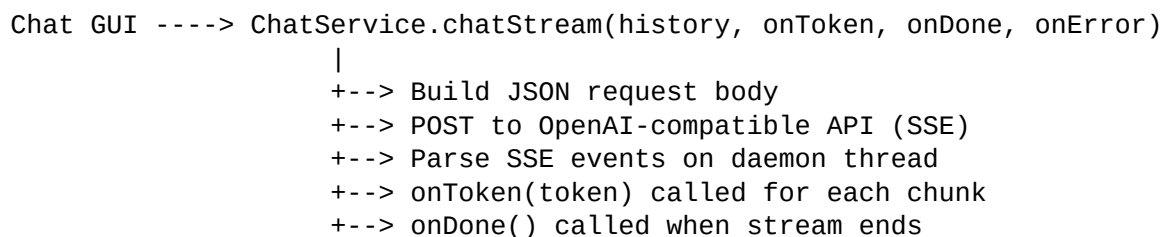
Step-by-step:

1. User clicks a button or interacts with a UI component
2. GUI event handler calls the appropriate DAO method
3. DAO obtains a Connection from DBConnection singleton
4. DAO creates a PreparedStatement with the SQL query
5. PreparedStatement is executed against the MariaDB server
6. ResultSet rows are mapped to POJO objects
7. DAO returns POJOs (or primitives) to the GUI
8. GUI updates its components with the returned data
9. Swing repaint cycle renders the updated UI

IMAGE LOADING FLOW (Async):



AI CHAT FLOW (Streaming):



+-> GUI updates chat display in real-time

DESIGN PATTERNS USED

Singleton

DBConnection uses the Singleton pattern to ensure a single shared database connection across all DAO classes.

Data Access Object (DAO)

Each entity has a dedicated DAO class that abstracts all database operations behind a clean API. This separates persistence logic from business logic and UI code.

Model-View (MV without Controller)

The architecture separates models (POJOs) from views (Swing GUIs). DAOs act as mediators that both models and views depend on.

MVC-like Separation

While not a strict MVC implementation, the layered architecture achieves similar separation of concerns:

Model -> POJO classes

View -> Swing GUI classes

Controller -> GUI event handlers (within GUI classes)

Service -> DAO classes

Lazy Loading + Cache-Aside

ImageCache implements lazy loading with a cache-aside pattern: check cache first, on miss load from disk/network, then populate cache.

Builder Pattern (Partial)

ChatService.Message uses Java records (effectively immutable builders) for clean message construction.

TECHNOLOGY STACK

Layer	Technology	Purpose
Presentation	Java Swing	Desktop GUI framework
Business Logic	Plain Java	DAO classes, chat service
Data Access	JDBC + MariaDB Connector	Database connectivity
Database	MariaDB / MySQL	Persistent storage
Utilities	Java stdlib (HttpClient, ImageIO, NIO)	Image caching, config, streaming HTTP
External Services:		
OpenAI API	- AI chat completions (GPT-4o-mini default)	
Image URLs	- Part images hosted on external servers	

DATABASE SCHEMA

Database: pc_build_store

Engine: InnoDB (supports foreign keys and transactions)

Tables:

- categories - 5 rows (CPU, GPU, RAM, Storage, PSU)
- parts - ~40 rows across all categories
- builds - User-created PC build configurations
- build_parts - Junction: builds <-> parts (one per category)
- gpu_options - GPU upgrade paths per budget tier
- bills - Finalized purchase records

See ER_Diagram.txt for complete entity-relationship documentation.